Comparison of VRP Solvers: Naive vs. Google OR-Tools

August 13, 2025

1 Introduction

This analysis compares a naive brute-force approach to solving the Vehicle Routing Problem (VRP) against Google's OR-Tools. The goal is to highlight the trade-offs between finding a provably exact solution and achieving a near-optimal solution efficiently.

2 Theoretical vs. Practical Optimality

The fundamental difference between the two solvers lies in their approach to optimality.

- **Naive Solver (Brute-Force):** This method explores every possible permutation of the path, guaranteeing that it will find the absolute shortest route. In a **theoretical sense**, this solution is 100% optimal because it has exhaustively proven that no shorter path exists. However, this approach is computationally expensive and becomes infeasible as the number of locations increases due to the factorial growth of possible paths.
- **Google OR-Tools:** OR-Tools uses a combination of heuristics and metaheuristics, such as **Guided Local Search**, to find a high-quality solution within a reasonable time. It starts with an initial feasible route and iteratively improves it. In a **practical sense**, the solution provided by OR-Tools is often considered optimal because the difference from the true optimum is negligible (e.g., 10^{-8} km), and the computational speed-up is significant. The slight discrepancy arises from its reliance on approximation algorithms rather than an exhaustive search.

3 Analysis of Results

Based on the provided data, we can directly compare the performance of both solvers for the same VRP instance with a waste facility at location 5.

3.1 Naive Solver

The naive solver found the following route and distance:

- **Optimal Path:** $5 \rightarrow 22 \rightarrow 57 \rightarrow 87 \rightarrow 70 \rightarrow 48 \rightarrow 5$
- **Minimum Distance:** 26.039 km
- **Processing Time:** 0.0006 seconds

This result represents the true, mathematically proven shortest path.

3.2 OR-Tools Solver

OR-Tools, given a 10-second time limit, found a different path with a very similar distance:

- **Optimal Path:** $5 \rightarrow 48 \rightarrow 70 \rightarrow 87 \rightarrow 22 \rightarrow 57 \rightarrow 5$
- **Minimum Distance:** 26.035 km
- **Processing Time:** 10.0193 seconds

Although the paths are different, the distance values are practically identical, with a minute difference of 0.004 km (4 meters).

4 Why OR-Tools May Not Find the Exact Optimum

Google's OR-Tools is designed for scalability and speed. It finds a good solution quickly, especially for large, complex problems where a brute-force approach would be impossible. The solver might not find the exact optimum for several reasons:

1. **Limited Search Time:** A constrained time limit prevents the solver from exploring the entire solution space. The 10-second limit in this example is sufficient for a good solution but not necessarily for a provably optimal one.
2. **Heuristic-Based Search:** The use of metaheuristics like **Guided Local Search** means the algorithm might get stuck in a "local optimum" — a solution that is better than its immediate neighbors but not the best overall solution.
3. **Initial Solution:** The quality of the final solution can be influenced by the initial route generated (e.g., using `PATH_CHEAPEST_ARC`), which might guide the search away from the absolute best path.

5 Conclusion

The comparison demonstrates the trade-off between **precision** and **performance**. The naive solver is an effective approach for its ability to find the absolute best solution, but its practical application is limited to very small problems. Google's OR-Tools offers a scalable and efficient way to find a high-quality, near-optimal solution for real-world VRP challenges, where a tiny difference in distance is negligible compared to the time saved.